Advanced Java
Complete Q&A Interview Guide

JDBC • Maven • HTTP • Servlet • Hibernate • Spring • Spring Boot

Total Questions: 55 | Topics: 7

JDBC

Q1.

What is the difference between Core Java and Advance Java?

• **Core Java (J2SE):** Used for desktop/standalone apps. Single tier. Covers basic fundamentals.

• **Advance Java (J2EE):** Used for web and enterprise server-side apps. Two-tier (client-server). Covers Servlet, JSP, Spring, Hibernate etc.

Q2.

What is JDBC?

JDBC stands for **Java Database Connectivity**. It is an API that allows Java applications to connect to a database (MySQL, Oracle, etc.) using a JDBC Driver to retrieve, insert, update, or delete data using SQL.

Q3.

What is a Database Driver?

A Database Driver is a software component that acts as a **bridge** between your Java application and the database. It enables Java to interact with and store data in the database.

Q4.

What are the 4 types of JDBC Drivers?

Type 1 – JDBC-ODBC Bridge: Windows-only, single-threaded. Removed in Java 7.

Type 2 – Native API Driver: Uses vendor-specific driver. No ODBC, but driver must be installed on every machine.

Type 3 – Network Protocol Driver: Driver installed on server; small copy sent to client via network. E.g. WebLogic RMI.

Type 4 – Thin/Universal Driver: Fully in Java, thread-safe, most widely used. E.g. mysql-connector jar, ojdbc jar.

Q5.

What are the steps of JDBC Connectivity?

1. Load & Register the Driver: **Class.forName("com.mysql.cj.jdbc.Driver")**
2. Get JDBC Connection via DriverManager (URL, user, password)
3. Create Statement or PreparedStatement
4. Execute query: executeQuery() for SELECT, executeUpdate() for INSERT/UPDATE/DELETE

5. Process ResultSet and close all connections

Q6.

What is DriverManager?

DriverManager is a class in java.sql package. It manages multiple JDBC drivers and provides the **getConnection()** method to establish a database connection. It acts as an interface between the user and the driver.

Q7.

What is Connection interface?

Connection (java.sql) maintains a session with the database. Used to create Statement/PreparedStatement objects and manage transactions via **commit()** and **rollback()**.

Q8.

What is Statement? What are its types?

Statement (java.sql) represents an SQL query and executes it against the database.

• Compiled & executed every time — for one-time queries only

• Cannot pass parameters at runtime

• Risk of SQL injection — low performance

• Best for DDL (CREATE, DROP, TRUNCATE)

Methods: execute(), executeQuery(), executeUpdate()

Q9.

What is PreparedStatement? Difference from Statement?

PreparedStatement extends Statement. It is a pre-compiled SQL query that can be reused multiple times with different parameters using '?'.

• PreparedStatement: Pre-compiled, faster, accepts runtime params, no SQL injection, used for DML (multiple times)

• Statement: Compiled every time, slower, no params, SQL injection possible, used for DDL (once)

Q10.

Difference between execute(), executeQuery(), executeUpdate()?

• **execute()** — For DDL (CREATE, DROP). Returns boolean.

• **executeQuery()** — For SELECT. Returns ResultSet.

• **executeUpdate()** — For INSERT, UPDATE, DELETE. Returns int (rows affected).

Q11.

What is ResultSet? What is RowSet?

ResultSet: Represents result of a SELECT query. Like a cursor — fetches rows one by one using next() (returns boolean). Always connected to DB. Cannot be serialized.

RowSet: Wrapper over ResultSet. More flexible. Can be connected or disconnected. Can be serialized.

Q12.

What is the purpose of Class.forName() method?

Class.forName("com.mysql.cj.jdbc.Driver") dynamically loads and registers the JDBC driver class into the JVM at runtime so DriverManager can use it to create a connection.

Maven

Q13.

What is Maven?

Maven is an open-source **project management and build tool** by Apache Software Foundation. Based on POM (Project Object Model). The word Maven means 'Accumulator of Knowledge'.

It helps: add JAR dependencies automatically, compile/test/package projects, and build .jar or .war files.

Q14.

How does Maven work? Explain its lifecycle.

Build Lifecycle Phases:

1. validate — Validates project & downloads dependencies
2. compile — Compiles source to .class files
3. test — Runs unit tests
4. package — Creates .jar or .war
5. install — Installs jar in local repo
6. deploy — Pushes jar to remote repo

Clean lifecycle deletes all generated files (.class, etc.)

Q15.

Difference between Maven and Gradle?

Maven: Uses XML (pom.xml), convention-based, enterprise standard, well-established.

Gradle: Uses Groovy/Kotlin (build.gradle), more flexible and faster, popular in Android development.

Q16.

What is the difference between local and remote repository?

Local Repository: Stored on your computer (.m2 folder). Maven checks here first.

Remote Repository: Hosted online (Maven Central). Maven downloads from here if dependency not found locally.

HTTP & Server

Q17.

What is HTTP? Why is it called stateless?

HTTP (HyperText Transfer Protocol) is a set of rules for communication between client (browser) and server.

It is **stateless** because once a request-response is complete, the connection is lost. The server does not remember the client for the next request.

Q18.

Explain HTTP Request and HTTP Response.

HTTP Request (Client → Server):

• Headers: Method (GET/POST/PUT/DELETE), URL, Content-Type, Cookies

• Body: Form data or JSON

HTTP Response (Server → Client):

• Headers: Status Code, Content-Type, Date

• Body: HTML, JSON data

Q19.

What are HTTP Status Codes?

• **2xx (200-299):** Success. e.g. 200 OK

• **3xx (300-399):** Redirection. e.g. 302 Found

• **4xx (400-499):** Client error. e.g. 404 Not Found, 403 Forbidden, 400 Bad Request

• **5xx (500-599):** Server error. e.g. 500 Internal Server Error

---

Q20.

What are HTTP Methods? Explain GET vs POST.

• **GET:** Fetches data. Data in URL (visible). Limited size. Can be bookmarked. Faster.

• **POST:** Sends data to server. Data in body (hidden). Large data. Cannot be bookmarked. Slower.

• **PUT:** Updates a record.

• **DELETE:** Deletes a record.

Q21.

What is a Server? What are its types?

A server is hardware/software that accepts and responds to client requests over a network.

Web Servers: Tomcat (Apache), IIS (Microsoft), Jetty, Glassfish

Application Servers: WebLogic, JBoss, IBM WebSphere, Wildfly

Note: One port can run only one application.

Q22.

How does a server-client application work?

1. Client sends HTTP request to web server
2. Server passes request to the corresponding Servlet
3. Servlet processes the request and generates a response
4. Response is sent back to client browser

5. Browser displays the output

Servlet

Q23.

What is a Servlet?

Servlet is a **server-side Java technology** used to handle client HTTP requests, process them, and generate dynamic responses (HTML, XML, JSON). Servlets are platform-independent and use HttpServletRequest and HttpServletResponse objects.

Q24.

Explain the Servlet Lifecycle.

1. **Loaded:** Servlet class is loaded by the container
2. **Instantiated:** Object is created (only once)
3. **Initialized:** init() is called once
4. **Service:** service() is called for every client request
5. **Destroyed:** destroy() is called when server shuts down

Q25.

Explain init(), service(), destroy() methods.

- **init():** Called once at first load. For initialization. Return type: void.
- **service():** Called for every request. Handles request and generates response. Return type: void.
- **destroy():** Called once when server shuts down. Cleans up resources. Return type: void.

Q26.

How many Servlet objects are created? When?

Only **one servlet object** is created — at the time of the very first request. The same object handles all subsequent requests (multithreaded).

Q27.

Difference between GenericServlet and HttpServlet?

GenericServlet: Protocol-independent (HTTP, FTP, SMTP). service() is abstract. No request/response objects.

HttpServlet: HTTP protocol only. service() is non-abstract. Has HttpServletRequest and HttpServletResponse objects.

Q28.

What is Servlet Collaboration?

Calling from one servlet to another servlet (or HTML file) is called **Servlet Collaboration**. Done using the RequestDispatcher object obtained from HttpServletRequest.

Q29.

What is RequestDispatcher? Difference from sendRedirect()?

RequestDispatcher (Interface):

- Internal redirection within the same application
- URL in browser does NOT change
- Methods: forward(req, res) and include(req, res)

sendRedirect() (HttpServletResponse method):

• External redirection to different app or URL

• URL in browser CHANGES

Q30.

What is web.xml / deployment descriptor?

web.xml is the **deployment descriptor file** placed in WEB-INF folder. It maps servlet class names to URL patterns.

In newer versions, it is replaced by the **@WebServlet** annotation directly on the class.

Q31.

What is @WebServlet annotation?

@WebServlet is a class-level annotation that maps a servlet to a specific URL pattern without needing a web.xml file. It is the modern replacement for the deployment descriptor.

Q32.

Difference between doGet() and doPost()?

• **doGet():** Handles HTTP GET requests. Used for retrieving data. Data in URL. Limited size.

• **doPost():** Handles HTTP POST requests. Used for submitting data. Data in body. Large data. More secure.

Hibernate

Q33.

What is ORM? What is Hibernate?

ORM (Object Relational Mapping) maps Java objects directly to database tables.

O = Object (POJO/Entity class), R = Relational (database), Mapping = bridge between them.

Hibernate is a Java ORM Framework that automatically maps entity objects to database tables using annotations.

Q34.

Difference between Hibernate and JDBC?

Hibernate: ORM framework, auto-maps objects, manages exceptions, uses HQL, easy association mapping, supports lazy loading, auto transaction management, 2-level cache (L1+L2).

JDBC: Raw API, manual object mapping, try-catch needed, uses SQL, difficult associations, no lazy loading, manual transactions, manual caching.

Q35.

What are the key components of Hibernate?

- **Configuration:** Loads settings from hibernate.cfg.xml
- **SessionFactory:** Creates Session objects. Long-lived. Thread-safe. Holds L2 Cache.
- **Session:** Represents a DB connection. Short-lived. Not thread-safe. Holds L1 Cache.
- **Transaction:** Manages begin, commit, rollback of DB transactions.

Q36.

What are Hibernate Annotations?

All from jakarta.persistence package:

- **@Entity** — Marks class as a DB table
- **@Id** — Marks the primary key field
- **@GeneratedValue(strategy=GenerationType.IDENTITY)** — Auto-increment for primary key
- **@Table** — Sets the database table name
- **@Column** — Sets the column name

Q37.

What are the States/Stages of objects in Hibernate?

- **Transient:** Object created but NOT associated with any session
- **Persistent:** Object associated with a Hibernate session (saved in DB)
- **Detached:** Object was persistent but session is now closed
- **Removed:** Object is marked for deletion from DB

Q38.

What is JPA?

JPA (Java Persistence API) is a **specification/set of rules** for ORM frameworks in Java.

It provides a common interface (persist(), merge()) that any ORM (Hibernate, TopLink, EclipseLink) can implement.

This lets you switch ORM frameworks without rewriting all your code.

Q39.

Difference between get() and load() in Hibernate?

get(): Returns null if not found. Always hits DB. Returns real object. Use when unsure if record exists.

load(): Throws ObjectNotFoundException if not found. Does NOT hit DB immediately (lazy). Returns proxy object. Use when sure record exists.

Q40.

Difference between Session and SessionFactory?

Session: One DB connection. Short-lived (per transaction). L1 Cache (default ON). NOT thread-safe.

SessionFactory: Creates Sessions. Long-lived (whole app). L2 Cache (default OFF). Thread-safe.

Q41.

What is HQL?

HQL (Hibernate Query Language) is an **object-oriented query language** similar to SQL but uses entity class names and properties instead of table/column names.

HQL is internally converted to SQL before execution, so SQL is slightly faster than HQL.

Q42.

What are Hibernate CRUD methods?

- **Insert:** save() / persist() — Return type: void
- **Update:** update() / merge() — Return type: void
- **Delete:** delete() / remove() — Return type: void
- **Fetch:** get() / load() — Return type: Object

Spring Core

Q43.

What is Spring Framework?

Spring is a **lightweight, open-source, loosely-coupled Java framework** for building enterprise applications. Developed by Rod Johnson in 2003. Has modules: IOC, AOP, MVC, DAO, ORM, etc. Uses annotations to avoid boilerplate code.

Q44.

What is IOC (Inversion of Control)?

IOC means the **Spring container creates and manages objects** for you — you don't use 'new' keyword directly.

You describe how objects should be created (annotations or XML), and Spring handles the rest.

Spring creates singleton objects by default (one instance shared throughout the application).

Q45.

What is Dependency Injection (DI)?

DI is the way IOC is implemented. Spring **injects dependencies** into your class automatically at runtime.

Objects are created before runtime and injected when needed. Makes code loosely coupled and easy to test.

Q46.

What is BeanFactory vs ApplicationContext?

Both are IOC containers.

BeanFactory: Basic container. Lazy initialization. Older approach.

ApplicationContext: Advanced container used in enterprise apps. Supports annotations, events, internationalization. Recommended for Spring applications.

Q47.

Explain Spring Annotations: @Component, @Autowired, @Qualifier, @Bean, @Scope

- **@Component** — Marks class as a Spring bean (auto-detected during component scan)
- **@Autowired** — Auto-injects a dependency (field, constructor, or method)
- **@Qualifier** — Specifies which bean to inject when multiple beans of same type exist
- **@Bean** — Method-level. Defines a bean inside a @Configuration class
- **@Scope** — Defines bean lifecycle: singleton (default) or prototype
- **@Value** — Injects a value into a field
- **@Configuration** — Marks class as source of bean definitions (replaces XML)
- **@ComponentScan** — Tells Spring which packages to scan for components

Q48.

What are AOP Annotations in Spring?

AOP = Aspect Oriented Programming. Used to add cross-cutting concerns (logging, security).

- **@Before** — Runs before the target method executes
- **@After** — Runs after the target method (like finally block — always executes)
- **@Around** — Surrounds the method; can choose whether to execute it or not. Most powerful.

Spring Boot

Q49.

What is Spring Boot?

Spring Boot is an **open-source framework built on top of Spring** that makes building Spring applications faster and easier. 'Boot' means rapid startup. Developed by Pivotal Software in 2014.

Includes embedded Tomcat server — no separate server setup needed.

Q50.

What are the core principles of Spring Boot?

- **Opinionated Defaults:** Pre-configured Maven, dependencies, project structure, application.properties
- **Convention over Configuration:** Uses annotations (@Controller, @Service, @Repository, @Entity)
- **Auto-Configuration:** Manages configuration automatically — no XML files needed
- **Embedded Server:** Built-in Tomcat — no separate setup

Q51.

What is the difference between REST and SOAP?

REST: Works on HTTP. Lightweight. Uses JSON (preferred). JSON is smaller and faster. Accessible from any internet device.

SOAP: Uses XML only. XML is heavier and needs more processing power. More device configuration required.

Q52.

What is JSON? How do you create a JSON object?

JSON (JavaScript Object Notation) stores data in key-value pairs.

```
{ "id": 1, "name": "Vishal", "city": "Pune" }
```

Array of objects:

```
[ { "id": 1, "name": "Vishal" }, { "id": 2, "name": "Rahul" } ]
```

Q53.

Explain Spring Boot Annotations.

- **@SpringBootApplication** — Main class. = @Configuration + @ComponentScan + @EnableAutoConfiguration
- **@RestController** — = @Controller + @ResponseBody. Used for REST APIs
- **@RequestMapping** — Maps HTTP requests to handler methods
- **@GetMapping / @PostMapping / @PutMapping / @DeleteMapping** — Shortcuts for HTTP methods
- **@Service** — Business logic layer
- **@Repository** — Data access layer (DAO)
- **@PathVariable** — Extracts value from URL: /users/{id}
- **@RequestBody** — Maps JSON request body to Java object
- **@RequestParam** — Extracts query param from URL: ?name=vishal

---

Q54.

What are the main Spring Boot dependencies?

- **Spring Web** — For building REST APIs

- **Dev Tools** — Hot reload during development
- **Spring Data JPA** — Database operations with Hibernate
- **Spring Boot Actuator** — App health monitoring
- **MySQL Driver** — Connect to MySQL database

Q55.

What are Stereotype Annotations in Spring Boot?

The four main stereotype annotations are:

- **@Component** — Generic Spring bean
- **@Service** — Business logic layer
- **@Repository** — Data access / DAO layer
- **@Controller** — Web layer (handles HTTP requests)